

GOBUSTER TEMELLERİ

[Kaynak](#)

İÇERİK

1. Hedef Ortam ve Ağ Konfigürasyonu
2. Giriş ve Temel Kavramlar
3. Gobuster: Directory and File Enumeration (Dizin ve Dosya Keşfi)
4. Web Enumeration ve Gelişmiş Sorun Giderme (Troubleshooting) Analizi
5. Subdomain Enumeration (Alt Alan Adı Keşfi)
6. Vhost Enumeration (Sanal Sunucu Keşfi)

Hedef Ortam ve Ağ Konfigürasyonu

Bu çalışmada hedefimiz, bir web sunucusu olarak yapılandırılmış **Ubuntu 20.04** sanal makinesidir (VM). Bu sunucu standart bir tekil yapıdan ziyade, üzerinde birden fazla alt alan adı (subdomain) ve **vhost** (Virtual Host - tek bir IP adresi veya fiziksel sunucu üzerinde birden fazla web sitesinin veya alan adının barındırılmasına olanak tanıyan web sunucu yapılandırması) barındırmaktadır. Ayrıca sunucu üzerinde iki farklı CMS (Content Management System - İçerik Yönetim Sistemi) kurulu durumdadır: **WordPress** ve **Joomla**. Bu çeşitlilik, hedef yüzeyin geniş olduğunu ve web dizinleri ile alt alan adlarının dikkatlice incelenmesi gerektiğini göstermektedir.

Gerçekleştirilecek **Enumeration** (Bilgi toplama - hedefin dizin yapısı, gizli dosyaları, alt alan adları ve servis konfigürasyonları hakkında haritalandırma yapmak amacıyla sistematik olarak veri elde etme süreci) aşamalarında temel aracımız **Gobuster** olacaktır. Gobuster, web sunucularındaki gizli yolları ve alt alan adlarını kaba kuvvet (brute-force) yöntemiyle tespit etmek için kullanılan, Go dili ile yazılmış son derece hızlı bir araçtır.

Dnsmasq Konfigürasyonu ve Yerel Alan Adı Çözümlemesi

ÖNEMLİ: Bu laboratuvar ortamında hedef ağ izole bir yapıdadır ve hedeflenen web sunucusu aynı zamanda ağın DNS (Domain Name System) sunucusu olarak görev yapmaktadır. Standart bir dış ağda çalışmadığımız için, tarayıcımızın veya Gobuster gibi araçlarımızın hedefin özel alan adlarını (örn. `site.local` gibi) çözebilmesi (resolve) için saldırı makinemizin DNS yapılandırmasını hedef makineye yönlendirmemiz kritik bir adımdır. Eğer bu konfigürasyon atlanırsa, yapılan hiçbir DNS sorgusu hedefe ulaşmayacak, araçlarımız hata verecek ve vhost tabanlı zafiyetler tespit edilemeyecektir.

Saldırı makinemizdeki DNS çözümleme işlemlerini yöneten Dnsmasq servisinin yapılandırma dosyasını düzenleyerek hedef IP adresini birinci isim sunucusu (nameserver) olarak atamamız gerekmektedir.

```
sudo nano /etc/resolv-dnsmasq
```

Yukarıdaki komut ile Dnsmasq servisinin yapılandırma dosyasını kök (root) yetkileriyle metin düzenleyicide açıyoruz.

Dosyanın en üst satırına aşağıdaki girdiyi eklemeliyiz:

```
nameserver 10.112.185.230
```

Bu satırın **en üste** (ilk satır olarak) yazılması hayati önem taşır. DNS çözümleme mekanizması yukarıdan aşağıya doğru çalışır. Eğer hedef IP adresimiz (**10.112.185.230**) ilk sırada olmazsa, sistem varsayılan DNS sunucusuna (örneğin Google DNS veya yerel ağ geçidi) sorgu atacak ve doğal olarak laboratuvar ortamındaki özel alan adlarını bulamayarak "NXDOMAIN" (Domain bulunamadı) hatası döndürecektir.

Değişiklikleri kaydedip (CTRL+O , ENTER , CTRL+X) çıktıktan sonra, yapılandırmanın sistem tarafından algılanıp devreye girmesi için Dnsmasq servisini yeniden başlatmamız gerekmektedir.

```
/etc/init.d/dnsmasq restart
```

Bu komut, arka planda çalışan DNS yönlendirme servisini durdurup yeni yapılandırma dosyası (/etc/resolv-dnsmasq) ile tekrar başlatır. Eğer bu komutu çalıştırmazsak, dosya değişmiş olsa bile işletim sistemi eski DNS ön belleği ve ayarları üzerinden çalışmaya devam eder.

DİKKAT: Yaptığımız yapılandırmanın doğruluğunu mutlaka teyit etmeliyiz. Sistem yöneticiliği ve siber güvenlikte varsayımlara yer yoktur.

```
cat /etc/resolv-dnsmasq
```

Yukarıdaki komut ile yapılandırma dosyasının içeriğini terminale yazdırarak kontrol ediyoruz.

```
root@tryhackme:~# cat /etc/resolv-dnsmasq
nameserver 10.112.185.230
nameserver 169.254.169.253
```

Terminal çıktısı incelendiğinde, hedef sistemimizin IP adresi olan **10.112.185.230**'un başarıyla birinci `nameserver` olarak ayarlandığı görülmektedir. İkinci satırda bulunan `169.254.169.253` adresi, saldırı makinesinin bulunduğu bulut ortamının (AWS vb.) yerel metadata veya varsayılan yedek DNS sunucusudur. Bizim eklediğimiz IP ilk sırada olduğu için, tüm yerel laboratuvar sorguları doğrudan hedef sunucuya gidecek, çözümlenemeyen

genel internet sorguları ise bir alt satırdaki sunucuya devredilecektir. Artık hedef üzerindeki vhost'lara ve alt alan adlarına yönelik sağlıklı bir **Enumeration** süreci başlatılabilir.

Giriş ve Temel Kavramlar

Gobuster'ın Anatomisi ve Siber Güvenlikteki Yeri

Gobuster, Golang programlama dili ile yazılmış açık kaynaklı, ofansif bir güvenlik aracıdır. Aracın temel amacı; hedef sistemlerdeki web dizinlerini, **DNS** alt alan adlarını (subdomains), sanal sunucuları (vhosts), Amazon **S3** bucket'larını ve Google Cloud Storage alanlarını tespit etmektir. Etik bilgisayar korsanlığı metodolojisinde bu araç, bilgi toplama (Reconnaissance) ve tarama (Scanning) fazları arasında kritik bir köprü görevi görür. Golang ile yazılmış olması, araca asenkron işlem yeteneği kazandırarak muazzam bir hız ve performans avantajı sağlar.

Sistemin derinliklerine inmeden önce, **Gobuster**'ın çalışma prensibini oluşturan iki temel kavramı netleştirmemiz gerekiyor:

Enumeration (Bir sistemdeki erişilebilir veya gizli kaynakların, dizinlerin, kullanıcıların veya alt alan adlarının sistematik olarak tespit edilip listelenmesi işlemi), hedef yüzeyin haritasını çıkarmak için elzemdir. Bu işlem, kaynakların yetkisiz kullanıcılara açık olup olmamasından bağımsız olarak gerçekleştirilir. **Brute Force** (Tüm olası kombinasyonların veya bir kelime listesindeki tüm değerlerin sırayla denenerek doğru eşleşmenin bulunması tekniği) ise, bu haritalandırmayı yapmak için kullandığımız motordur. **Gobuster**, önceden hazırlanmış kelime listelerini (**wordlist**) kullanarak hedefe eşzamanlı binlerce istek gönderir ve dönen HTTP yanıtlarına göre kaynağın var olup olmadığını doğrular.

Temel Komut Yapısı ve Analizi

Gobuster, Kali Linux gibi penetrasyon testi dağıtımlarında varsayılan olarak yüklü gelir. Aracın sunduğu modülleri ve parametreleri anlamak için yardım menüsünü incelemek, duruma uygun konfigürasyon yapabilmemiz için ilk adımdır. Aşağıdaki komut ile aracın yetenek haritasını görüntülüyoruz:

```
gobuster --help
```

Bu komutu çalıştırdığımızda, aracın temel iskeletini, desteklediği saldırı modüllerini ve genel yapılandırma bayraklarını gösteren aşağıdaki standart çıktı ile karşılaşırız:

Alt Komutlar (Available Commands) Analiz Tablosu

Komut (Command)	Teknik Derinlik ve Operasyonel İşlevi
dir	Hedef web uygulamasındaki gizli dizinleri ve dosyaları tespit eden ana Enumeration modudur. Sunucuya gönderilen HTTP GET/HEAD

Komut (Command)	Teknik Derinlik ve Operasyonel İşlevi
	isteklerine dönen Durum Kodlarını (Status Codes) analiz ederek hedefin arka planındaki dosya ağacını haritalandırır.
dns	Hedef alan adına (Domain) ait alt alan adlarını (Subdomain) bulmaya yarar. DNS sunucularına eşzamanlı Brute Force sorguları yollayarak, dışarıdan görünmeyen geliştirici ortamlarını (dev.example.com) veya iç ağa bakan yönetim panellerini açığa çıkarır.
vhost	Aynı IP adresi üzerinde barındırılan ancak dışarıdan bilinmeyen Sanal Sunucuları (Virtual Hosts) hedefler. Araç, HTTP isteklerindeki Host: başlığını manipüle eder ve sunucunun yapılandırma bazlı farklı bir yanıt (Routing) verip vermediğini test eder.
fuzz	Geleneksel taramanın yetersiz kaldığı uç durumlar için Fuzzing motorudur. URL'in, HTTP başlıklarının (Headers) veya istek gövdesinin (Request Body) herhangi bir yerine FUZZ anahtar kelimesi yerleştirilir ve kelime listesindeki veriler (Payloads) doğrudan bu noktaya enjekte edilir.
s3	Amazon AWS bulut mimarisindeki S3 birimlerini (Buckets) tarar. Bulutta yetkilendirmesi yanlış yapılandırılmış (Misconfigured) ve dışarıya açık bırakılmış depolama alanlarını tespit ederek hassas veri sızıntılarını (Data Leakage) yakalamayı hedefler.
gcs	s3 komutuyla aynı mantıkta çalışarak, operasyonu Google Cloud Storage (GCS) mimarisindeki bucket'lara yönelik gerçekleştirir.
tftp	UDP üzerinden çalışan TFTP (Trivial File Transfer Protocol) sunucularını tarar. TFTP yetkilendirme gerektirmediği için, sunucuda unutulmuş kritik konfigürasyon veya yedek (Backup) dosyalarını çekmek için kullanılır.
completion	Terminal kabuğunda (Bash, Zsh) otomatik tamamlama (Tab tuşu) betiğini (Script) üretir. Zamanın kritik olduğu sertifika sınavlarında (Örn: OSCP) hızı ve operasyonel verimliliği artırır.
version / help	Aracın mevcut sürümünü veya yardım menüsünü yazdırır. Özellikle bağımlılık çakışmaları veya versiyon kaynaklı "False Positive" sonuçları teşhis etmek (Troubleshooting) için sürüm kontrolü önemlidir.

Operasyonel Parametreler (Flags) Analiz Tablosu

Parametre (Flag)	Teknik Derinlik, Mantık ve Acil Durum Kullanımı
-t, --threads	Eşzamanlı açılacak iş parçacığı (Thread) sayısını belirler (Varsayılan: 10). ÖNEMLİ: Çok yüksek değerler taramayı aşırı hızlandırır ancak hedef sunucuda DoS etkisi yaratabilir veya WAF/IPS gibi güvenlik cihazlarının anormal trafiği anında tespit edip IP adresinizi bloklamasına (Ban) yol açar.
-w, --wordlist	Brute Force işleminde denenecek mühimmatı (Örn: SecLists) barındıran dosya yolunu belirtir. Eğer yol yerine - işareti verilirse,

Parametre (Flag)	Teknik Derinlik, Mantık ve Acil Durum Kullanımı
	veriler standart girdiden (STDIN) okunur; bu sayede farklı araçların çıktıları Gobuster'a anında borulanabilir (Piping).
--wordlist-offset	Taramaya wordlist'in belirli bir satırından (Örn: 500000) devam etmeyi sağlar. Ağ bağlantısı koptuğunda veya sistem çöktüğünde operasyona sıfırdan başlamak için kullanılan hayati bir B Planı parametresidir.
--delay	Modern güvenlik sistemlerini atlatmak (Evasion) için kullanılır. İstekler arasına milisaniye (Örn: 1500ms) bazında kasıtlı bekleme süresi koyarak, saldırı trafiğini normal bir kullanıcının veya Google botunun gezinme hızına benzetir (Rate Limiting atlatma).
-o, --output	Terminalde görünen değerli bulguları (Status 200, 301 vb.) standart çıktıya ek olarak kalıcı bir dosyaya yazdırır. Raporlama aşamasında kanıt (Evidence) sunmak veya çıktıları otomatize bir şekilde Nmap gibi diğer araçlara beslemek için mecburidir.
-p, --pattern	Standart wordlist okumasının ötesine geçerek, bir dosyadaki desenleri (Patterns) kelimelere uygular. Örneğin; hedefte yedek dosyası arıyorsanız, kelimelerin sonuna dinamik olarak .bak , .old , .tar.gz ekleyerek varyasyonlar üretilmesini sağlar.
-q, --quiet	Terminaldeki banner (Gobuster logosu ve versiyonu) ile gereksiz bilgilendirme mesajlarını tamamen gizler. Çıktıların temiz kalmasını ve sadece hedeflenen teknik bulgulara odaklanılmasını sağlar.
-v, --verbose	Çıktı detay seviyesini maksimuma çıkarır. Sadece başarılı olanları değil, yönlendirmeleri (HTTP 3xx), izin verilmeyenleri (HTTP 403) ve diğer tüm ara bulguları tam hedeflerle birlikte loglar. Kapsamlı analiz için kritik öneme sahiptir.
--debug	Operasyon arka planındaki teknik logları ekrana basar. Eğer araç sürekli zaman aşımı (Timeout) hatası veriyorsa veya beklenmedik bir şekilde duruyorsa, sorunun kök nedenini (Root Cause) teşhis etmek için açılır.
--no-error	Sunucu tarafı iletişim hatalarını (Örn: Connection Refused, Timeout) ekrana yazdırmayı engeller. Hedefte yoğun bir IPS koruması varsa ve sürekli hata dönüyorsa, terminal kirliliğini önleyerek sadece çalışan isteklere odaklanmayı sağlar.
-z, --no-progress / --no-color	İlerleme çubuğunu veya renkli terminal çıktısını devre dışı bırakır. Amacı estetik değildir; ekran çıktılarını grep , awk veya sed gibi araçlarla regex kullanarak parse ediyorsanız, gizli terminal kaçış (Escape) karakterlerinin scriptlerinizi bozmasını engeller.

Çıktı bize aracın tek bir dümdüz araç olmadığını, alt komutlar (commands) üzerinden modüler bir yapıda çalıştığını gösterir. Örneğin; **dir** (Dizin ve dosya taraması), **dns** (Alt alan adı taraması) ve **vhost** (Aynı IP üzerindeki sanal sunucu taraması) gibi farklı enumeration modları sunar. Doğru operasyon için doğru modun seçilmesi zorunludur.

Kritik Parametrelerin (Flags) Derinlemesine İncelemesi

Sektörde hiçbir zaman bir aracı sadece varsayılan ayarlarıyla çalıştırmayız. Hedefin savunma mekanizmalarına (IPS/WAF) ve ağ durumuna göre parametreleri manipüle etmeliyiz. Yardım menüsünde yer alan en hayati parametreleri şu şekilde inceliyoruz:

-t, --threads parametresi, arka planda eşzamanlı olarak açılacak ve istek gönderecek iş parçacığı sayısını belirler. Varsayılan değeri 10'dur ve devasa boyutlardaki bir **wordlist** için bu sayı oldukça yavaştır. İşlemci gücümüz ve ağ bant genişliğimiz elverdiği ölçüde bu değeri artırarak operasyon süresini kısaltabiliriz. **ÖNEMLİ:** Gereğinden yüksek thread sayıları, hedef web sunucusunda yük yaratıp Denial of Service (DoS) etkisi oluşturabilir veya siber güvenlik cihazları tarafından anormal trafik olarak algılanıp IP adresimizin derhal banlanmasına yol açabilir.

-w, --wordlist parametresi, **Brute Force** işlemi sırasında sistemin baz alacağı deneme listesinin dosya yolunu belirtir. Aracın kalbi burasıdır; komut içindeki her bir satır alınır, hedef URL'in yapısına entegre edilir ve istek yapılır. Yanlış hedefe yanlış **wordlist** seçmek, saatlerce sürecektir anlamsız bir trafikten başka bir şey üretmez.

--delay parametresi, her bir iş parçacığının yeni bir istek göndermeden önce bekleyeceği süreyi belirler. **DİKKAT:** Modern savunma sistemleri, saniyeler içinde gelen yüzlerce anlamsız isteği "Rate Limiting" kurallarına takarak engeller. Bu mekanizmaları atlatmak (Evasion) ve trafiğimizi normal bir kullanıcı veya Google botu gibi göstermek için bu parametre ile kasıtlı gecikmeler ekleriz (Örn: --delay 1500ms).

--debug parametresi, operasyon sırasında arka planda çalışan ve varsayılan olarak gizlenen tüm detaylı logları ekrana yansıtır. Eğer tarama beklenmedik bir şekilde takılıyor, sunucu hataları veriyor veya bağlantı düşüyorsa, sorunun kök nedenini (Root Cause) saptamak için bu bayrağı aktif ederek sorunu teşhis ederiz.

-o, --output parametresi, ekrana yansıyan anlamlı sonuçları (bulunan dizinler, dosyalar vb.) belirlediğimiz bir dosyaya kaydeder. Raporlama aşamasında kanıt sunabilmek veya daha sonra diğer araçlarla (Örn: Nmap, Nikto) bu dizinleri otomatize tarayabilmek için çıktıları mutlaka loglamalıyız.

Gerçek Dünya Senaryosu: Dizin Taraması (Directory Enumeration) Analizi

Tüm bu kavramları ve parametreleri bir araya getirerek gerçek bir sızma testi senaryosunu adım adım okuyalım. Aşağıdaki komut, tipik bir dizin tarama operasyonudur:

```
gobuster dir -u "http://www.example.thm/" -w /usr/share/wordlists/dirb/small.txt -t 64
```

Bu komutu yazdığımızda arka planda işleyen teknik mantık sırasıyla şudur:

İlk olarak **gobuster dir** ifadesi ile araca, hedefin HTTP protokolü konuşan bir web sunucusu olduğunu ve klasör/dosya yapısını hedefleyen "dir" modunda çalışması gerektiğini iletiyoruz.

Ardından **-u "http://www.example.thm/"** parametresi ile taramanın yapılacağı kök URL'i belirtiyoruz. Araç tüm yolları bu temel adresin üzerine inşa edecektir.

Sonrasında **-w /usr/share/wordlists/dirb/small.txt** parametresi ile **Brute Force** için kullanacağımız sözlüğü sisteme veriyoruz. **Gobuster**, **small.txt** dosyası içindeki her bir değeri alır ve hedefe yollar. Eğer wordlist'in ilk satırında "images" kelimesi varsa, araç <http://www.example.thm/images/> adresine bir HTTP GET isteği gönderir. Eğer sunucu "200 OK" dönerse bu dizinin var olduğunu; "403 Forbidden" dönerse dizinin var olduğunu ama erişim yetkimizin olmadığını; "404 Not Found" dönerse böyle bir dizin olmadığını anlarız.

Son olarak **-t 64** parametresi ile eşzamanlı istek kapasitesini 10'dan 64'e çekiyoruz. Bu agresif bir yaklaşımdır ve "Hızlı sonuç istiyorum, hedef ağ buna dayanabilir" mesajını taşır.

Plan B: Eğer bu komutu çalıştırdığımızda ekranda sürekli zaman aşımı (Timeout) hataları görseydik veya bağlantımız tamamen kopsaydı, güvenlik cihazları tarafından tespit edildiğimizi anlardık. Böyle bir durumda komutu durdurup; thread sayısını düşürecek (**-t 5**), istekler arasına bekleme süresi koyacak (**--delay 2000ms**) ve kullanıcı ajanı (User-Agent) başlığını sıradan bir tarayıcı gibi manipüle ederek daha gizli (stealth) bir yaklaşım izleyecektik.

Gobuster: Directory and File Enumeration (Dizin ve Dosya Keşfi)

Dizin Yapılarının Anatomisi ve Tespit Mantiği

Bir sızma testi (Penetration Test) sırasında, hedef web uygulamasının mimarisini ve iskeletini anlamak en kritik adımdır. **Gobuster**'ın **dir** (Directory) modu tam olarak bu noktada devreye girer. Çoğu web uygulaması ve içerik yönetim sistemi (CMS), tahmin edilebilir ve standartlaştırılmış klasör hiyerarşilerine sahiptir. Bu öngörülebilirlik, hedefin **Brute Force** ataklarına karşı savunmasız kalmasını sağlar.

Örneğin, tipik bir WordPress kurulumunun barındırıldığı sunucudaki dizin ağacına (Directory Tree) bakalım:

```
root@tryhackme:~# tree -L 3 -d.  
├── html  
│   └── wordpress  
│       ├── wp-admin  
│       ├── wp-content  
│       └── wp-includes
```

Yukarıdaki ağaç yapısı bize şunu anlatır: Dışarıdan bir saldırgan, `wp-admin` veya `wp-content` gibi yaygın izin isimlerini içeren bir kelime listesi (**wordlist**) kullandığında, arka planda yatan bu dosya hiyerarşisini kolayca haritalandırabilir. **Gobuster**, hedefe bu yolları (Paths) sorar ve dönen **HTTP Status Codes** (HTTP Durum Kodları) üzerinden bizim adımıza bu yapının erişilebilirliğini doğrular. 200 OK (Erişilebilir), 403 Forbidden (Var ama yetkisiz) veya 301 Redirect (Başka yere yönlendirilmiş) gibi kodlar, sistemin karanlık noktalarını aydınlatan fenerlerimizdir.

İleri Seviye 'dir' Modu Parametreleri (Flags)

dir modu, hedef sistemin savunma mekanizmalarına göre ince ayar yapmamızı sağlayan spesifik parametrelere sahiptir. Yardım menüsünü (`gobuster dir --help`) incelediğimizde karşımıza çıkan en kritik bayrakların teknik analizleri aşağıdaki tabloda verilmiştir:

Parametre (Flag)	Teknik Derinlik, Mantık ve Operasyonel Kullanımı
<code>-c, --cookies</code>	Hedef sisteme gönderilen her isteğe oturum (Session) çerezlerini ekler. Eğer hedefte yetkilendirme (Authentication) gerektiren, sadece giriş yapmış kullanıcıların görebildiği bir portal varsa (<code>PHPSESSID=...</code> gibi), bu bayrak sayesinde kimliği doğrulanmış (Authenticated) bir Enumeration gerçekleştirilir.
<code>-x, --extensions</code>	Sadece izinleri değil, spesifik uzantılara sahip dosyaları da arar. Örneğin <code>.php</code> , <code>.js</code> , <code>.bak</code> , <code>.txt</code> . Hedef sistemde unutulmuş bir konfigürasyon dosyası (<code>config.php.bak</code>) veya ifşa olmuş bir JavaScript dosyası ararken operasyonun kalbini oluşturur.
<code>-H, --headers</code>	HTTP isteklerine özel başlıklar (Headers) enjekte eder. WAF atlama (Evasion) tekniklerinde, API anahtarı (Authorization: Bearer...) gönderimlerinde veya hedef sunucuyu farklı bir User-Agent ile manipüle etmek gerektiğinde zorunludur.
<code>-k, --no-tls-validation</code>	Hayati Parametre: Hedef HTTPS kullanıyorsa ve sertifikası kendi imzaladığı (Self-Signed) geçersiz bir sertifikaysa, araç güvenlik uyarısı verip taramayı durdurur. CTF'lerde, TryHackMe/HackTheBox laboratuvarlarında veya iç ağ sızma testlerinde bu sertifika doğrulama sürecini atlamak ve taramayı zorla devam ettirmek için kullanılır.
<code>-n, --no-status</code>	Ekrana basılan sonuçlarda HTTP durum kodlarını gizler. Terminal çıktısını temiz tutmak ve sadece bulunan yolların saf bir listesini elde etmek istendiğinde tercih edilir.
<code>-P, --password / -U, --username</code>	Hedefte Basic Authentication (Temel Kimlik Doğrulama) penceresi (Örn: htaccess koruması veya Tomcat yönetim paneli) varsa, önceden elde ettiğimiz veya tahmin ettiğimiz kimlik bilgilerini kullanarak tarama yapmamızı sağlar. Birlikte kullanılmalıdır.
<code>-s, --status-codes</code>	Sadece bizim belirlediğimiz HTTP durum kodlarını (Whitelist) döndüren sonuçları gösterir (Örn: <code>200,204,301,302,307,401,403</code>). Diğer tüm çöpleri (Örn: 404 Not Found) filtreler.

Parametre (Flag)	Teknik Derinlik, Mantık ve Operasyonel Kullanımı
<code>-b, --status-codes-blacklist</code>	Belirtilen durum kodlarını yok sayar (Blacklist). Plan B Kullanımı: Bazı kötü yapılandırılmış web sunucuları, olmayan bir dizin sorulduğunda 404 yerine 200 OK (False Positive) döner (Örn: Özel hata sayfaları). Bu durumda her kelime bulunmuş gibi görünür. Bu bayrakla o yanıltıcı kodu veya boyutu engelleyerek taramayı kurtarırız. <code>-s</code> bayrağını ezer.
<code>-r, --followredirect</code>	Sunucu 301 veya 302 yönlendirme kodu döndüğünde, aracın o yönlendirmeyi takip ederek nihai hedefe (Location başlığındaki adrese) gitmesini sağlar. Gizli bir dizinin bizi farklı bir domaine veya alt dizine taşıyıp taşımadığını görmek için elzemdir.

Operasyonel Senaryolar ve Mantıksal Analiz

Teoriyi pratiğe dökelim. Standart bir dizin tarama operasyonu şu sözdizimiyle başlatılır:

```
gobuster dir -u "http://www.example.thm" -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -r
```

Bu komutu çalıştırdığımızda terminalde olanların arka plan açıklaması:

- dir** : Aracın "dizin ve dosya keşfi" motorunu ateşler.
- u "http://www.example.thm"** : Hedefin Kök Dizinini (Root Web Directory) belirler (Linux Apache sunucularda genellikle `/var/www/html` dizinine tekabül eder).
 - ÖNEMLİ DETAY 1 (Protokol):** URL mutlaka `http://` veya `https://` protokolü ile başlamalıdır. Protokol girilmezse araç paketleri nasıl yönlendireceğini bilemez ve tarama anında başarısız (Fail) olur.
 - ÖNEMLİ DETAY 2 (IP vs Hostname):** URL kısmına doğrudan hedefin IP adresini (Örn: `http://10.10.10.5`) yazmak cezbedici olsa da tehlikelidir. Tek bir sunucu (Tek IP), **Virtual Hosting (VHOST)** mimarisi sayesinde onlarca farklı web sitesi barındırabilir. IP adresi kullanırsanız, Apache'nin varsayılan (Default) sayfasına düşersiniz. Kesin hedefe ulaşmak için her zaman Hostname (`example.thm`) kullanılmalıdır.
 - ÖNEMLİ DETAY 3 (Recursive Tarama):** Gobuster varsayılan olarak **Özyinelemeli (Recursive)** DEĞİLDİR. Eğer bu tarama sonucunda `/resources` adında gizli bir dizin bulursanız, araç o dizinin içine girip alt klasörleri taramaz. Orayı taramak için komutu `-u "http://www.example.thm/resources"` şeklinde manuel olarak tekrar başlatmanız gerekir.
- w [...]directory-list-2.3-medium.txt** : Taramada kullanılacak mühimmat dosyasını belirtir. Wordlist içindeki her satır (Örn: `admin`), kök URL'in sonuna eklenir (`http://www.example.thm/admin`) ve hedefe fırlatılır.
- r** : Yönlendirmeleri takip et. Eğer `/admin` dizini 301 döndürüp bizi `/login.php` 'ye yönlendiriyorsa, araç bu zinciri izler ve nihai sonucu bize bildirir.

Dosya Uzantısı Fuzzing (Extension Enumeration)

Sadece klasör bulmak bir sızma testi için yeterli değildir; asıl hedef bilgi ifşasıdır (Information Disclosure). Aşağıdaki komut, yaklaşımımızı daha agresif hale getirir:

```
gobuster dir -u "http://www.example.thm" -w  
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x .php,.js
```

Bu varyasyonda eklenen `-x .php,.js` parametresi, aracın çalışma mantığını geometrik olarak katlar. Gobuster artık kelime listesinden aldığı "login" kelimesi için hedefe 3 farklı istek atar:

1. `http://www.example.thm/login` (Klasör arar)
2. `http://www.example.thm/login.php` (PHP dosyası arar - Arka uç mantığı)
3. `http://www.example.thm/login.js` (JavaScript dosyası arar - İstemci tarafı mantığı ve olası API rotaları)

Mantık Zinciri: Bu adım başarılı olursa, sistemde indekslenmemiş dosyaları keşfederiz. Örneğin bir `database.php` veya eski bir geliştiriciye ait `test.js` dosyası bulunursa, bir sonraki aşamada bu dosyaların içeriklerini analiz ederek (Source Code Review) kritik zafiyetler veya doğrudan sunucu şifreleri elde edebiliriz.

Web Enumeration ve Gelişmiş Sorun Giderme (Troubleshooting) Analizi

Ağ Katmanı ve DNS Çözümleme (Resolution) Engellerinin Aşılması

Bir sızma testi (Penetration Test) operasyonunun en kritik ve genellikle en çok hata yapılan ilk aşaması, hedef sistemle doğru ağ iletişimini kurmaktır. Operasyonun başında **Gobuster** aracı çalıştırıldığında alınan `no such host` (Böyle bir makine yok) hatası, hedefin bir IP adresi değil, iç ağa (Intranet) özel bir alan adı (`www.offensivetools.thm`) olmasından kaynaklanmıştır. Global DNS sunucuları bu adresi tanımaz.

Bu engeli aşmak için işletim sisteminin yerel adres defteri olan `/etc/hosts` dosyasına müdahale edilmesi gerekir. Ancak standart bir kullanıcı olarak bu dosyaya `echo` komutuyla veri yazmaya çalışıldığında sistem güvenliği devreye girerek `permission denied` (Erişim Engellendi) hatası fırlatır. Bu bir **Privilege Escalation** (Yetki Yükseltme) sorunudur.

Sorunu kökünden çözmek ve dinamik olarak atanan hedef IP adresini (Target IP) sisteme tanıtmak için tam yetkili bir kök kabuğu (Root Shell) açılarak yönlendirme yapılmalıdır:

```
sudo sh -c 'echo "10.114.162.193 www.offensivetools.thm" >> /etc/hosts'
```

Yukarıdaki komut çalıştırıldığında sistem şu mantıkla eğitilir: "Dış dünyadaki DNS sunucularına gitme; `www.offensivetools.thm` adresine yönelik tüm paketleri doğrudan `10.114.162.193` IP adresine yönlendir." Hedef makine her yeniden başlatıldığında (Restart) IP adresi değişeceğinden, bu DNS eşleme (Mapping) işlemi yeni IP ile tekrarlanmalıdır.

Dizin Keşfi (Directory Enumeration) ve Kaynak Tüketimi (Timeout) Analizi

Ağ bariyeri aşıldıktan sonra hedefin dosya hiyerarşisini çıkarmak için **Gobuster** devreye sokulur. Ancak operasyon sırasında terminalin kırmızı `timeout occurred during the request` (istek sırasında zaman aşımı oluştu) hatalarıyla dolması, hedefin çöktüğünün (Denial of Service) habercisidir.

DİKKAT: Lab ortamlarındaki veya zayıf konfigürasyonlu sunucularda yüksek **Thread** (iş parçacığı) kullanmak (`-t 20` veya üstü), sunucunun HTTP isteklerine yanıt verememesine yol açar. Böyle bir durumda hedef makine yeniden başlatılmalı (Restart) ve tarama hızı düşürülerek daha sinsi (Stealth) bir yaklaşım izlenmelidir:

```
gobuster dir -u "http://www.offensivetools.thm" -w  
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -t 10
```

Bu temel tarama sonucunda birçok 200 OK durum kodlu standart dizin (Örn: `/images` , `/components`) bulunurken, siber güvenlik perspektifinden en çok dikkat çeken (catch attention) dizin `/secret` olmuştur. Sunucunun bu dizin için **403 Forbidden** (Yasaklı/Erişim Yok) yanıtı dönmesi, içeride yetkisiz kullanıcıların görmemesi gereken hassas verilerin saklandığına dair çok güçlü bir kriptografik işarettir.

Hedefli Fuzzing (Extension Enumeration) ve Dosya Tespiti

Ana dizinlerin haritası çıkarıldıktan sonra, tespit edilen gizli `/secret` dizinine odaklanılarak ikinci ve daha agresif bir saldırı dalgası (Fuzzing) başlatılmalıdır. Amaç, klasörleri değil, hedef sistemde unutulmuş spesifik kod dosyalarını bulmaktır.

```
gobuster dir -u "http://www.offensivetools.thm/secret" -w  
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x .js
```

Bu komutun mimarisini incelediğimizde; `-u` parametresi ile hedefin Base URL (Temel Adres) kısmı doğrudan şüpheli dizine (`/secret`) kilitlenmiştir. `-x .js` parametresi ise araca kelime listesindeki her girdinin sonuna JavaScript uzantısını ekleyerek tarama yapmasını emreder. Bu spesifik operasyonun çıktısı siber güvenlik analizi açısından kusursuzdur:

```
/flag.js (Status: 200) [Size: 22]
```

Bu satır, `/secret` dizini altında doğrudan erişilebilir (Status: 200) ve sadece 22 byte boyutunda (Size: 22) bir dosya olduğunu kesinleştirir. Dosya boyutunun küçüklüğü, içinde karmaşık bir web sayfası değil, muhtemelen düz metin (Plain Text) formatında aranan bayrağın (Flag) bulunduğunu doğrular.

Veri Sızdırma (Data Exfiltration) ve Bağlamsal Yol (Path Context) Mantığı

Dosya tespit edildikten sonra içeriğini terminale çekmek (Exfiltration) için `curl` aracı kullanılır. Ancak operasyon sırasında `curl http://www.offensivetools.thm/flag.js` komutunun devasa bir 404 (Page Not Found) HTML sayfası döndürmesi ve `grep` filtrelerinin boşa çıkması, sektörde sıkça düşülen "Bağlamsal Yol (Path Context)" tuzağıdır.

ÖNEMLİ: Gobuster, taramayı `/secret` dizini altında yapmıştır. Dolayısıyla bulunduğu `/flag.js` dosyası ana dizinde değil, o şüpheli klasörün içindedir. Hatalı URL ile istek atıldığında web sunucusu (Joomla CMS) böyle bir dosya bulamadığı için kendi özel 404 hata sayfasının HTML kodlarını geri döndürür. Veriyi başarıyla çekmek ve filtreleme ihtiyacı duymadan doğrudan terminale basmak için mutlak yol (Absolute Path) kullanılmalıdır:

```
curl -s http://www.offensivetools.thm/secret/flag.js
```

Bu nihai komuttaki `-s` (Silent) parametresi, dosya indirilirken terminalde oluşan ilerleme çubuklarını (Progress Bar) ve gereksiz gürültüyü engelleyerek tamamen temiz bir veri akışı sağlar. Yanlış dizin tuzağı aşıldığında ve doğru URL hedeflendiğinde, 22 byte boyutundaki hedeflenen "THM{...}" bayrağı hiçbir ekstra HTML veya JS kodu barındırmaksızın doğrudan operasyon ekranına düşer ve süreç %100 başarıyla tamamlanır.

Subdomain Enumeration (Alt Alan Adı Keşfi)

Saldırı Yüzeyinin Genişletilmesi ve Hedef Mantığı

Sızma testlerinde (Penetration Testing) savunma mekanizmaları genellikle ana alan adına (Örn: `tryhackme.thm`) odaklanacak şekilde inşa edilir. Güvenlik duvarları (WAF), yama yönetimleri ve erişim kontrolleri bu ana karargahta kusursuz olabilir. Ancak devasa bir kurumun veya uygulamanın altyapısında, genellikle unutulmuş, yaması eksik veya eski bir teknolojiyle çalışan alt alan adları (Subdomains) bulunur (Örn: `dev.tryhackme.thm` veya `mobile.tryhackme.thm`).

Gobuster'ın `dns` modu tam olarak bu zayıf halkaları bulmak için tasarlanmıştır. Bu işlem sadece bir liste çıkarma işi değildir; ana hedeften sızamadığımız durumlarda, hedefin arka bahçesine açılan yamalı olmayan bir kapı (Vulnerability Exploitation) bulma operasyonudur.

'dns' Modu Operasyonel Parametreleri (Flags)

Dizin keşfi (`dir`) moduna kıyasla `dns` modu daha az parametre barındırır, çünkü arka planda çalışan protokol HTTP değil, doğrudan DNS sorgularıdır (UDP/TCP Port 53). Ancak

bu parametrelerin her biri kritik istihbarat sağlar.

Parametre (Flag)	Teknik Derinlik, Mantık ve Acil Durum Kullanımı
<code>-d, --domain</code>	Zorunlu Parametre: Taranacak hedef kök alan adını (Root Domain) belirtir. Aracın, kelime listesindeki verileri neyin önüne ekleyeceğini bilmesi için şarttır (Örn: <code>example.thm</code>).
<code>-c, --show-cname</code>	Hedefin çözümlendiği (Resolve) CNAME (Canonical Name) kayıtlarını ekrana basar. Teknik Anlamı: Bulduğumuz bir alt alan adı aslında başka bir servise (Örn: AWS, Heroku, Github Pages) işaret ediyorsa bunu görürüz. Siber güvenlikte bu bilgi, Subdomain Takeover (Alt Alan Adı Devralma) zafiyetlerini avlamak için altın değerindedir. (<code>-i</code> parametresi ile aynı anda kullanılamaz).
<code>-i, --show-ips</code>	Bulunan alt alan adlarının çözümlendiği arka plandaki gerçek IP adreslerini de listeler. Mantık Zinciri: Eğer ana domain bir WAF (Örn: Cloudflare) arkasındaysa ve <code>104.x.x.x</code> dönüyorsa, ancak bulduğumuz <code>dev.example.thm</code> alt alan adı doğrudan gerçek sunucu IP'sini (<code>192.168.x.x</code> veya farklı bir Public IP) dönüyorsa, WAF'ı tamamen atlatmış (Bypass) oluruz. Tüm saldırı trafiğini artık bu IP üzerinden yürütürüz.
<code>-r, --resolver</code>	İşletim sisteminizin varsayılan DNS sunucusunu kullanmak yerine, sorguları sizin belirlediğiniz özel bir DNS sunucusuna (Örn: <code>8.8.8.8</code> veya <code>1.1.1.1</code>) yönlendirir. Plan B: Eğer ISP'niz (İnternet Servis Sağlayıcınız) çok hızlı DNS sorguları attığınız için sizi engellerse veya hedefin kendi iç DNS sunucusuna (Zone Transfer denemesi sonrası) spesifik sorgular atmanız gerekirse bu bayrak hayat kurtarır.

Operasyonel Senaryo ve Komut İskeleti

Brute Force yöntemiyle alt alan adlarını taramak için, dizin taramasından farklı olarak hedefte bir kelime listesi (Wordlist) kullanırız. Ancak bu liste klasör isimlerini değil, `admin` , `mail` , `dev` , `shop` gibi yaygın alt alan adı önklerini barındırır.

```
gobuster dns -d example.thm -w /usr/share/wordlists/SecLists/Discovery/DNS/subdomains-top1million-5000.txt
```

Bu komutu çalıştırdığımızda işletilen teknik mantık şudur:

- `gobuster dns` : Aracı HTTP/HTTPS katmanından çıkarıp doğrudan DNS katmanına geçiririz.
- `-d example.thm` : Hedef alan adını kilitleriz.
- `-w [...]subdomains-top1million-5000.txt` : DNS keşfi için endüstri standardı olan **SecLists** mühimmatını kullanırız. Araç listenin ilk satırındaki `all` kelimesini alır, `all.example.thm` adresini oluşturur ve DNS sunucusuna sorar: *"Bu adrese ait bir A veya CNAME kaydı var mı?"*

Terminal Çıktısının Analitik Yorumu

Yukarıdaki komut hedefe gönderildiğinde alınan başarılı çıktı aşağıdaki gibidir:

```
=====
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Domain:      example.thm
[+] Threads:     10
[+] Timeout:     1s
[+] Wordlist:     /usr/share/wordlists/SecLists/Discovery/DNS/subdomains-
top1million-5000.txt
=====
Starting gobuster in DNS enumeration mode
=====
Found: www.example.thm
Found: shop.example.thm
Found: academy.example.thm
Found: primary.example.thm
Progress: 4989 / 4990 (99.98%)
=====
Finished
=====
```

Kök Neden Analizi (Root Cause Analysis): Bu çıktı sadece "4 tane site bulduk" demek değildir; siber güvenlik uzmanına bir sonraki saldırı adımlarını dikte eder:

- **shop.example.thm** : Burada muhtemelen bir e-ticaret altyapısı (Örn: Magento, WooCommerce) çalışmaktadır. Bir sonraki adımda bu spesifik adrese odaklanıp ödeme sistemleri veya kullanıcı veri tabanları üzerinde SQL Injection (SQLi) veya XSS zafiyetleri aranmalıdır.
- **academy.example.thm** : Bir e-öğrenme platformu (Örn: Moodle) barındırıyor olabilir. Varsayılan kimlik bilgileri (Default Credentials) veya yetki yükseltme (Privilege Escalation) zafiyetleri için ideal bir hedeftir.
- **primary.example.thm** : İsminden dolayı kurumun iç işleyişine veya ana sunucu (Primary Server) veritabanı düğümlerine (Node) işaret ediyor olabilir. Geliştiriciler bazen bu tür yapıları dışarıya açık unuturlar.

Bir Sonraki Operasyonel Adım: Bu DNS taramasından elde edilen alt alan adlarının her biri ayrı birer hedeftir. Bir önceki konuda öğrendiğimiz **gobuster dir** (Dizin Taraması) komutu, şimdi bu yeni bulunan alt alan adlarının (Örn: `http://shop.example.thm`) her biri için ayrı ayrı çalıştırılarak hedefin tüm anatomisi gün yüzüne çıkarılmalıdır.

Vhost Enumeration (Sanal Sunucu Keşfi)

Vhost (Virtual Host) Nedir ve DNS'ten Farkı Nerededir?

Bir sızma testinde en sık karıştırılan iki kavram "Subdomain" (Alt Alan Adı) ve "Vhost"tur (Sanal Sunucu). Yüzeyde ikisi de `dev.example.thm` gibi görünür ancak arka plandaki mimarileri ve tespit edilme yöntemleri tamamen farklıdır.

- Subdomain (DNS Modu):** Bu yapılar DNS sunucularında fiziksel olarak bir A (IPv4) veya AAAA (IPv6) kaydına sahiptir. Sorgu DNS sunucusuna yapılır (`8.8.8.8` 'e "Bu adresin IP'si ne?" diye sorulur). DNS modu bunu arar.
- Vhost (Vhost Modu):** Tek bir fiziksel sunucu ve **tek bir IP adresi** (Örn: `192.168.1.100`), Apache veya Nginx'in "Routing" (Yönlendirme) yetenekleri sayesinde içinde onlarca farklı web sitesi barındırabilir. Dış dünyada DNS kaydı olmayabilir. Sunucu, kendisine gelen HTTP isteğindeki **Host: başlığına (Header)** bakarak hangi sitenin kodlarını çalıştıracığına karar verir.

Mantık Zinciri: Eğer hedef kurumun iç ağına sızdıysanız veya DNS sunucuları dış dünyaya kapalıysa (`Zone Transfer` başarısızsa), DNS modu işe yaramaz. Çünkü DNS kayıtlarına ulaşamazsınız. İşte bu noktada Vhost modu devreye girer. Arka planda DNS sunucusuna değil, doğrudan hedefin IP adresine (`-u`) HTTP istekleri atarız. Her istekte **HTTP Host: başlığını** manipüle ederek ("Sende `admin.example.thm` var mı?", "Peki `dev.example.thm` var mı?") sunucunun farklı bir site açıp açmadığını test ederiz.

'vhost' Modu Operasyonel Parametreleri (Flags)

Vhost taraması, HTTP protokolü üzerinden yapıldığı için `dir` moduna çok benzer parametreler kullanır.

Parametre (Flag)	Teknik Derinlik ve Operasyonel Mantık
<code>-u, --url</code>	Zorunlu Parametre: HTTP isteklerinin gönderileceği hedef IP adresini (veya Hostname'i) belirtir. (Örn: <code>http://10.114.162.193</code>). Bu, paketlerin fiziksel rotasıdır.
<code>--domain</code>	HTTP Host: başlığındaki ana (Root) alan adını tanımlar (Örn: <code>example.thm</code>).
<code>--append-domain</code>	Kelime listesinden (Wordlist) alınan kelimeleri (<code>admin</code> , <code>dev</code>) doğrudan yollamak yerine, <code>--domain</code> ile belirttiğimiz alan adını kelimenin sonuna ekler (<code>admin.example.thm</code>). DNS altyapısının olmadığı test lab'larında (CTF'lerde) zorunludur. Aksi takdirde sunucu sadece Host: admin görür ve ne demek istediğinizi anlamaz.
<code>--exclude-length</code>	Filtreleme Hayat Kurtarıcısı: Bazen hedef sunucu, var olmayan bir Vhost sorulduğunda standart bir hata sayfası döndürür (Status 200 dönebilir veya özel bir 404 sayfası olabilir). Bu durum her denemenin "Bulundu" (False Positive) görünmesine sebep olur. Bu hata sayfalarının boyutu genelde sabittir (Örn: 279 byte). Bu bayrakla (<code>--exclude-length 250-</code>

Parametre (Flag)	Teknik Derinlik ve Operasyonel Mantık
	320 veya spesifik boyut 279) o boyuttaki cevapları çöpe atarak sadece gerçek siteleri yakalarız.
-m, --method	İstek tipini değiştirir (Varsayılan: GET). Bazen WAF'lar GET isteklerini izlerken POST isteklerine kör kalabilir veya bir API sadece POST isteğiyle Vhost'u açığa çıkarabilir.

Operasyonel Senaryo ve İleri Seviye Komut Analizi

Gerçekçi bir sızma testi (veya TryHackMe laboratuvar) ortamında, DNS altyapısı tam oturmadiğı için karmaşık bir komut kurgulamak zorundayız. Aşağıdaki komut tam bir "Master Class" operasyonudur:

```
gobuster vhost -u "http://10.114.162.193" --domain example.thm -w /usr/share/wordlists/SecLists/Discovery/DNS/subdomains-top1million-5000.txt --append-domain --exclude-length 250-320
```

Bu devasa komutun arka planda nasıl bir HTTP isteği oluşturduğuna bakalım:

```
GET / HTTP/1.1
Host: [WORDLIST_KELİMESİ].example.thm <-- Manipüle edilen kısım burasıdır
User-Agent: gobuster/3.6
Connection: keep-alive
```

Adım Adım Kod Çözümü:

1. `vhost -u "http://10.114.162.193"` : Paketlerin fiziksel olarak gideceği hedef IP adresidir.
2. `-w [...]subdomains-top1million-5000.txt` : Denenecek prefix'leri (admin, dev, shop) barındıran SecLists mühimmatı.
3. `--domain example.thm --append-domain` : Kelime listesindeki "shop" kelimesini al, sonuna ".example.thm" ekle ve HTTP isteğinin içine `Host: shop.example.thm` olarak yerleştir.
4. `--exclude-length 250-320` : Eğer dönen cevabın (Response Body) boyutu 250 ile 320 byte arasındaysa bu varsayılan hata sayfasıdır (False Positive), bunu ekrana yazma. (Kritik: Eğer komutu bu bayrak olmadan çalıştırırsanız, terminaliniz binlerce sahte "Found" mesajıyla kilitlenecektir.)

Terminal Çıktısı ve Yorumlama

Komut başarıyla çalıştığında karşımıza şu şekilde temiz ve rafine bir liste çıkar:


```
Found: blog.example.thm Status: 200 [Size: 1493]
Found: shop.example.thm Status: 200 [Size: 2983]
Found: www.example.thm Status: 200 [Size: 84352]
Found: academy.example.thm Status: 200 [Size: 434]
```

Saldırı Vektörü (Next Steps):

IP adresi aynı (10.114.162.193) olmasına rağmen, HTTP Host: başlığını değiştirerek farklı sitelerin kapılarını açtık. DNS kayıtlarında bu siteler dışarıya sızdırılmamış olabilir ama sunucunun içinde yaşıyorlar.

Bir sonraki adımda, tarayıcımızdan shop.example.thm adresine gidip içerideki web uygulamasını incelemek isteyeceğiz. Ancak unutmayın, bilgisayarımız (AttackBox/Kali) bu ismin hangi IP'ye ait olduğunu bilmez. Tarayıcıda açabilmek için yine **B Planı** devreye girer: /etc/hosts dosyasını açıp tespit ettiğimiz bu Vhost'ları hedefin IP adresine manuel olarak eşlemeliyiz:

```
# /etc/hosts dosyasının içine eklenecek satır:
10.114.162.193 www.example.thm shop.example.thm blog.example.thm
academy.example.thm
```

Eşleme yapıldıktan sonra hedefleri web tarayıcısında (veya Burp Suite'te) açıp SQLi, XSS veya bilgi ifşası zafiyetleri aramaya başlayabiliriz.

```
Dataview (inline field
'=====
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Domain:      example.thm
[+] Threads:     10
[+] Timeout:     1s
[+] Wordlist:
/usr/share/wordlists/SecLists/Discovery/DNS/subdomains-
top1million-5000.txt
=====
Starting gobuster in DNS enumeration mode
=====
Found: www.example.thm
Found: shop.example.thm
Found: academy.example.thm
Found: primary.example.thm
Progress: 4989 / 4990 (99.98%)
=====
```

Finished

=====')

: Error:

-- PARSING FAILED -----

> 1 |

=====

| ^

2 | Gobuster v3.6

3 | by OJ Reeves (@TheColonial) & Christian Mehlmauer
(@firefart)

Expected one of the following:

('(', 'null', boolean, date, duration, file link, list ('[1, 2,
3]'), negated field, number, object ('{ a: 1, b: 2 }'), string,
variable